

CONTEXTGRAPH: REPAIR MEMORY FOR MISSING REQUIREMENTS

Anonymous authors

Paper under double-blind review

ABSTRACT

Past repairs record requirements that a new issue may leave unstated. We study repair memory as a way to supply these missing requirements during code generation. On a Django development task, an ordinary repair and an additional issue-based review pass official verification but discard query parameters during redirection. Supplying a repair of the same API enables the writer to preserve both the required deployment prefix and the omitted query parameters. Providing that history in the first call produces the same code logic as a later history-guided revision, using one target repair call instead of two. A second development case in pytest reaches both method-level and class-level skip requirements in one history-informed repair call, where the observed ordinary sequence needed a diagnostic revision. An executed source relation identifies the missing Django behavior, but its delivery yields the same revision as the original historical patch. The evidence supports a simple procedure: give the writer the relevant historical requirement and evaluate whether the resulting repair preserves it together with the current requirement.

1 INTRODUCTION

The issue description tells a coding agent what is currently reported as broken. It need not enumerate every behavior a complete repair should preserve. A past repair can supply a requirement absent from the new issue, allowing memory to improve a repair that already passes the target’s tests.

Consider an administrative redirect that must preserve a deployment prefix. Changing its destination from `request.path_info` to `request.path` fixes that reported fault. However, appending a slash to the resulting path still discards query parameters. A different repair of the same API records this omitted behavior and replaces path concatenation with `request.get_full_path(force_append_slash=True)`. The useful information in this experience is the requirement to retain the query, which must hold together with the deployment-prefix requirement.

This view leads to a direct repair procedure: supply the current issue and a relevant historical repair before generating code. We compare this procedure with supplying the same experience after an initial repair, and with an additional review using only the current issue. The comparison determines whether the corrective information requires a separate revision stage.

2 USING HISTORY TO COMPLETE THE REPAIR REQUIREMENTS

Supply the requirement during generation. The repair writer receives the target issue, initial implementation, and selected source issue and patch. The historical record describes an additional behavior for the new repair to preserve. The writer returns a repaired method adapted to the target implementation while preserving the method’s public signature and decorator. The procedure uses one target repair call. Our experiments supply a source repair of the same API to study how its information changes the result.

Measure the requirements together. We immediately evaluate every generated patch with the official SWE-bench verifier (Jimenez et al., 2024). To measure the behavior contributed by the source

Table 1: Django 14404 development comparison. Every row passes official verification: 2 fail-to-pass and 323 pass-to-pass tests. Calls include the initial generation when a revision is used.

Procedure	Calls	Prefix	Query	Joint
Issue only	1	Pass	Fail	Fail
Issue + ordinary review	2	Pass	Fail	Fail
History in revision	2	Pass	Pass	Pass
Executed relation in revision	2	Pass	Pass	Pass
History in first call	1	Pass	Pass	Pass

repair, a separate writer generates a short check from its patch and relevant repaired code. A normal execution changing from false on the buggy source to true on the repaired source demonstrates the historical behavior on the supplied inputs. We then execute this check unchanged on the patched target. An issue check measures the reported requirement, and a joint input exercises it together with the historical requirement. These are measurement programs; the direct repair writer receives only the issue, initial code, and raw history.

3 EVIDENCE: A HISTORICAL REQUIREMENT COMPLETES THE REPAIR

We use the training side of a SWE-bench Verified split with 280 source tasks, 70 development tasks, and a separate 150-task evaluation partition unused by these experiments. The development comparison pairs source Django 16612 with target Django 14404. The target issue asks for support of `FORCE_SCRIPT_NAME` in administrative redirects; the source records query preservation in the same method.

An ordinary repair omits the query. DeepSeek V4 Pro generates one initial repair from the issue and relevant initial code, including the available request-path APIs. The writer may replace `AdminSite.catch_all_view()` while preserving its signature and decorator. Its repair keeps URL resolution based on `path_info` and constructs the redirect from `request.path`. Official verification passes, and the issue check confirms that the deployment prefix is retained. The source check executes normally on this candidate and observes that both query strings are discarded.

History supplies the corrective information. Three independent revision calls start from this candidate and receive either the issue-based evidence alone, the historical issue and patch, or the executed source relation. The issue-based review retains the initial patch. Both memory conditions replace path concatenation with `get_full_path(force_append_slash=True)`, producing byte-identical patches that preserve the prefix and query. All calls use the same model, temperature 0.7, and a 16,384-token output allowance.

The information can be supplied in the first call. We then append the identical historical issue and patch to the original initial-generation prompt, without supplying a candidate or diagnostic feedback. This one-call procedure produces a method with the same AST as the history-guided revision and passes the same checks (Table 1). The observed generation uses 4,482 provider-reported tokens, compared with 13,601 for the original initial call and historical-patch revision combined. For this development case, the additional revision stage contributes no behavioral improvement over supplying history initially.

Both requirements hold on a joint input. An independently prepared check combines the deployment prefix with repeated query parameters and percent-encoded values: `keep=a%2Fb&keep=c%20d`. The input is withheld from all writers and executed after their official verifications. Both history-guided revisions and the direct history generation preserve the exact prefix and query string; the initial and issue-reviewed candidates lose the query string.

A second application avoids a failed initial repair. The public issue in Pytest 7236 concerns a skipped test method whose `tearDown()` still runs under `-pdb`. Source 10081 records the corresponding class-level skip requirement. One call supplied with this history adapts the source helper

to the target’s existing `__unittest_skip__` flags and checks both the method and its containing class. It passes official verification (1 fail-to-pass and 51 pass-to-pass tests), both skip checks, and a joint session that also checks a normal test’s lifecycle. The earlier ordinary initial call fails; one issue-only diagnostic revision then repairs both skip conditions using the runtime skip state. Direct history reaches these measured behaviors in the first repair call, using 8,346 provider-reported tokens compared with 25,883 for that observed two-call sequence.

4 RELATED WORK

ExpeL distills reusable insights from experience, and Getafix learns applicable edit patterns from human repairs (Zhao et al., 2024; Bader et al., 2019). ExpeRepair uses historical test demonstrations and reflective insights to generate reproduction and boundary tests (Mu et al., 2026). MIRA constructs repair memory from SQL corrections and uses database evidence to activate and combine applicable repair units (Liu et al., 2026). STAIR abstracts trajectories into hierarchical plans supplied before repair, including guidance that completes partial fixes (Xu et al., 2026). Jiang et al. (2026) study late requirement disclosure in coding-agent sessions and controlled tasks. Our development comparison examines when the compact source issue and patch suffice to preserve the historical and current requirements together, and whether an additional revision or executed-relation input changes the result.

AI USE STATEMENT

Generative AI assisted literature review, method development, implementation, experiments, analysis, and writing.

REFERENCES

- Johannes Bader, Andrew Scott, Michael Pradel, and Satish Chandra. Getafix: Learning to fix bugs automatically, 2019. URL <https://arxiv.org/abs/1902.06111>.
- Bowen Jiang, Haowei Cheng, Yuhong Fu, Anne Koziolok, Jialong Li, and Weixing Zhang. Requirements after the first edit: Mining late requirement emergence and rework in real-world coding-agent sessions, 2026. URL <https://arxiv.org/abs/2609.03028v1>. Version 1.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues?, 2024. URL <https://arxiv.org/abs/2310.06770>.
- Yining Liu, Chenyu Yang, Boyan Li, Rui Mao, and Yuyu Luo. MIRA: Evidence-verified repair memory for text-to-sql correction, 2026. URL <https://arxiv.org/abs/2608.06950v1>.
- Fangwen Mu, Junjie Wang, Lin Shi, Song Wang, Shoubin Li, and Qing Wang. ExpeRepair: Dual-memory enhanced LLM-based repository-level program repair, 2026. URL <https://arxiv.org/abs/2506.10484v2>. Version 2; accepted at FSE 2026.
- Yisen Xu, Jiayuan Zhou, Ruiqi Pan, and Tse-Hsun Chen. Reusing past repairs through hierarchical trajectory abstraction for coding agents, 2026. URL <https://arxiv.org/abs/2607.29658v1>. Version 1.
- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: Llm agents are experiential learners, 2024. URL <https://arxiv.org/abs/2308.10144v3>.